

Server-Side Rendering (SSR) Differences with PWA Kit

PWA Kit and Storefront Next implement SSR, but with fundamentally different architectures that reflect their respective technology stacks. PWA Kit's SSR approach gives developers explicit control through Express and React Query, with optional partial hydration via Islands. Storefront Next uses React Router 7's built-in SSR with automatic streaming, middleware chains, and loader-based data fetching. Storefront Next's approach reduces boilerplate and provides better streaming performance, while the PWA Kit approach offers more granular control over hydration timing.

Contents

Key Differences

- Data Flow Comparison
- Hydration Approaches
- Storefront Conversion Considerations

Key Differences

Aspect	PWA Kit	Storefront Next
SSR Runtime	Express server with <code>runtime.render</code>	React Router 7 framework mode
Data Hydration	React Query cache serialization	Promise streaming with <code>useSuspenseQuery</code> hook
Middleware	Express middleware	React Router middleware chain
Partial Hydration	<code>Island</code> component with strategies	Suspense boundary with <code>createIsland</code>
Server Context	<code>useServerContext()</code> hook	Context middleware chain

[Try for free](#)

Splitting	Route auton splitti
Cache Control	Manual via <code>res.set()</code> Via H' respo from middl

Data Flow Comparison

PWA Kit:

1. Express receives request.
2. React tree renders.
3. React Query hooks execute during render.
4. Data fetched, components re-render with data.
5. Query cache serialized to HTML.
6. Client hydrates with cached data.

Storefront Next:

1. React Router receives request.
2. Middleware chain runs (auth, config, i18n).
3. Loaders execute, return promises.
4. React renders, `use()` unwraps promises.
5. HTML streams as promises resolve.
6. Client hydrates, `clientLoader` runs.

Hydration Approaches

PWA Kit offers fine-grained control with `Island`:

```
1 <Island hydrateOn="visible">
2   <Footer />
3 </Island>
```

Storefront Next uses Suspense boundaries:

```
1 export default createPage({
2   component: ProductDetail,
3   fallback: <ProductSkeleton />,
4 });
```

[Try for free](#)

side logic like setting cache headers moves from component hooks to middleware or loaders.

2. React Query vs. Loaders: Data fetching moves from hooks inside components to loader functions exported from route files.
3. Island vs. Suspense: Partial hydration with Island becomes Suspense boundaries with `createPage`.
4. Express routes vs. Resource routes: Custom API endpoints move from Express handlers to React Router resource routes.

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblazer Comm
Events and Calen
Partner Communi
Blog
Salesforce Admin
Salesforce Archite

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#)

[Use of Cookies](#) [Cookie Preferences](#) [Your Privacy Choices](#)

